Michigan Technological University

Michigan Technological University
Digital Commons @ Michigan Tech

Michigan Tech Publications

1-9-2026

Jiving with LLMs: Assessing First Year Students' Computer Programming Self Efficacy After Reading Code With LLMs

Michelle Jarvie-Eggart
Michigan Technological University, mejarvie@mtu.edu

Joseph Roy Teahen
Michigan Technological University, jrteahen@mtu.edu

Daniel T. Masker
Michigan Technological University, dtmasker@mtu.edu

Jose Padilla
Michigan Technological University, padilla@mtu.edu

Leo C. Ureel II
Michigan Technological University, ureel@mtu.edu

Follow this and additional works at: https://digitalcommons.mtu.edu/michigantech-p2


see next page for additional authors
Part of the Computer Sciences Commons, and the Engineering Commons



This work is licensed under a Creative Commons Attribution 4.0 International License.

Recommended Citation

Jarvie-Eggart, M., Teahen, J., Masker, D. T., Padilla, J., Ureel, L. C., Brown, L., Pomerville, S., & Sticken, J. (2026). Jiving with LLMs: Assessing First Year Students' Computer Programming Self Efficacy After Reading Code With LLMs. *Journal of Teaching and Learning with Technology, 14*(1). http://doi.org/10.14434/jotlt.v14i1.41701
Retrieved from: https://digitalcommons.mtu.edu/michigantech-p2/2685

Follow this and additional works at: https://digitalcommons.mtu.edu/michigantech-p2


Part of the Computer Sciences Commons, and the Engineering Commons

Authors

Michelle Jarvie-Eggart, Joseph Roy Teahen, Daniel T. Masker, Jose Padilla, Leo C. Ureel II, Laura E. Brown, Scott Pomerville, and Jon Sticken

This article is available at Digital Commons @ Michigan Tech: https://digitalcommons.mtu.edu/michigantech-p2/2685

Journal of Teaching and Learning with Technology, Vol. 14, Special Issue, pp.65-87.
doi: 10.14434/jotlt.v12i1.41701

Jiving With LLMs: 1st-Year Students' Computer Programming Self-Efficacy After Reading Code with LLMs

Michelle Jarvie-Eggart
Michigan Technological University
mejarvie@mtu.edu

Joseph Roy Teahen
Michigan Technological University

Daniel T. Masker
Michigan Technological University

José Padilla
Michigan Technological University

Leo C. Ureel II
Michigan Technological University

Laura E. Brown
Michigan Technological University

Scott Pomerville
Northern Michigan University

Jon Sticken
Michigan Technological University

Abstract: This study investigated the impact of leveraging generative artificial intelligence (GenAI) to assist 1st-year engineering and computer science (CS) students in reading code in a new (to them) language. Students were asked to comment code in FORTRAN. They were then asked to run the code through ChatGPT-4.0 for its comments and reflect on what they learned from the experience. Participants completed survey items from Ramalingam and Wiedenbeck's Computer Programming Self-Efficacy Scale (CPSES) prior to and after the intervention. Additional open-ended reflective (qualitative) questions were added to the quantitative questions in the postintervention questionnaire. This study documents increases in self-efficacy for programming independence and persistence (Factor 1 of the CPSES) as well as for complex programming tasks (Factor 2) after students used ChatGPT for generating code explanations. Both CS and engineering students showed improvements in programming independence and persistence; but only engineers showed significant improvements in their confidence regarding complex programming tasks. Men experienced a significant increase in self-efficacy on Factor 1 while women experienced a significant increase on Factor 2. The qualitative data point to an increase in student understanding of the new code and suggest that although students may be more likely to use GenAI for assistance as they progress through programming courses, guiding students in using GenAI to understand code may shift students' intent away from using GenAI to write code for them. Thus, we recommend that programming faculty instruct students how to interact with GenAI.

Keywords: novice programming, introductory programming, generative AI, LLM, code explanations, computer programming self-efficacy

Computer programming is becoming an essential skill for all workers (Winnie, 2021). Yet, attrition rates are notoriously high in introductory programming courses (Margulieux et al., 2020). The rapid advancement of generative artificial intelligence (GenAI) tools capable of producing source code is transforming education (Geng et al., 2023), providing an opportunity for educators to leverage. Chatbots can make programming more approachable to novices and those in disciplines beyond computer science (CS) and engineering. Chatbots such as ChatGPT facilitate interactions between humans and computers via natural language (Shawar & Atwell, 2007) and generate code across multiple languages (OpenAI, 2025). GPT-4 can solve all homework assignments in Michigan Technological University's (MTU's) introductory programming sequence. Such capabilities (Finnie-Ansley et al., 2022; Geng et al., 2023; Kiesler & Schiffner, 2023) have caused faculty to fear that students will misuse GenAI rather than leverage it as a learning tool (Denny et al., 2024; Petricini et al., 2024). However, faculty believe students will inevitably use GenAI (Petricini et al., 2024; Wermelinger, 2023). Thus, faculty must change their approach to teaching programming (Jacques, 2023) and teach students to use GenAI critically (Styve et al., 2024). We believe moderate and critical use of GenAI among students learning to code can enhance their understanding, knowledge, and skill. Our approach emphasizes cognitive and procedural programming aspects before code implementation to ensure foundational skill development.

Literature Review

The most important facilitating condition for the adoption of new technologies, including programming languages, is other people (Jarvie-Eggart et al., 2021, 2024). Unfortunately, other people are not always available to help novice programmers. GenAI is always available to provide quick, correct answers to student programming questions (Kasneci et al., 2023; Lo, 2023; Yilmaz & Karaoglan Yilmaz, 2023a). Incorporating AI in education enables personalized learning experiences, providing timely and relevant feedback (Bond et al., 2024; Holstein et al., 2019; Roll & Wylie, 2016), fostering metacognitive awareness, and encouraging self-regulated learning (Koedinger et al., 2013). AI can automate routine administrative tasks, allowing educators to focus on the human-centered aspects of teaching, mentorship, motivation, and classroom discussions (Luckin et al., 2016).

AI Use in Programming Education

Because platforms such as ChatGPT answer prompts in natural language, even students without programming experience can use them to create simple programs (Yilmaz & Karaoglan Yilmaz, 2023b). GenAI can make programming more approachable to novices, resulting in higher programming scores (Kazemitabaar et al., 2023). GenAI also increases 1st-year programming students' understanding of basic programming concepts and reduces their anxiety (Reunanen & Nieminen, 2024). Scholl and colleagues (2024) found that problem understanding was the main use of ChatGPT for 1st-year computing students. Yilmaz and Karaoglan Yilmaz (2023a, 2023b) found that ChatGPT improved students' thinking skills, helped with debugging and self-confidence, supported teaching of computational thinking, increased creativity, improved both algorithmic and critical thinking, improved problem solving, increased cooperativity, and increased student motivation.

Cautions for AI Use

Although some faculty have expressed concerns over student GenAI use on account of hallucinations, this type of misinformation is relatively rare when assisting novice-level coders (Kuramitsu et al., 2023). The challenge most frequently mentioned by researchers regarding AI use in education is that it has no ethical component (Bond et al., 2024). Students have this concern as well as faculty (Diaz et al., 2025). Research also highlights concerns regarding teachers' lack of AI literacy (Bond et al., 2024), and the majority of faculty desire training about AI (Mah & Groß, 2024).

Additionally, using GenAI may increase students' "metacognitive laziness" (Fan et al., 2025), as students can use it to offload cognitive responsibility, particularly high-order Bloom taxonomy tasks (Handa et al., 2025). Over-reliance on GenAI affects student programmers' critical thinking, creativity, and problem solving (Deriba et al., 2024). Thus, students should receive guidance regarding critical, intentional, and moderate use to aid their learning instead of replacing their thinking.

Programming Skill Development

Language syntax and mechanics lie lower in Bloom's taxonomy than problem solving in programming (Edwards et al., 2018; Ullah et al., 2020). Novice programmers need to balance learning the syntax of their first few languages with learning more foundational concepts, such as tracing code and developing abstract reasoning (Fowler et al., 2022). Code-tracing abilities support the development of the ability to explain code, which develops before complex code-writing abilities (Lister et al., 2006, 2009; Sheard et al., 2008; Venables et al., 2009) and is a foundational skill for programmers (Cunningham et al., 2022; Lister et al., 2009; Murphy et al., 2012; Sarsa et al., 2022). Following this progression, students learn code tracing, writing simple syntax, comprehending code templates, and writing code with templates in sequence (Xie et al., 2019).

Code explanations are essential scaffolding for understanding how snippets of code execute. Code explanations can also decrease the stress of learning for novice programmers (Griffin, 2016). Students perceive code explanations generated by a large language model (LLM) as more accurate and easier to understand than student-generated comments (Leinonen et al., 2023). Leinonen et al. asserted that AI can be helpful scaffolding for students to be able to understand code explanations while not yet able to generate their own accurately. Additionally, code explanations created by GenAI can facilitate at-scale instruction in programming courses (MacNeil et al., 2023).

Leveraging AI to Explain Code

In developing programming skills, students must learn to explain their own code. This is an essential skill for debugging made infamous in "rubber ducking" (Hunt & Thomas, 1999), yet it remains a task many find difficult (Zi et al., 2025). Provided code explanations can reduce the cognitive load of debugging (McCauley et al., 2008). AI can produce code explanations; however it should be used with caution as the explanations can have inaccuracies (Sarsa et al., 2022).

Kiesler and Schiffner (2023) found that both ChatGPT-3.5 and GPT-4 provided code explanations for 97.3% of all test cases, but that lower quality code produced lower quality explanations. Similarly, Herden (2024) found that ChatGPT-3.5's explanations were accurate only with well-written code and recommended its use for explaining solutions to students. Banjade et al. (2024) found that GPT-4, ChatGPT-3.5, Mixtral, and Llama-2 created 100% correct code explanations for Java. Studies comparing GenAI comments to those of experts found that the AI-generated comments were correct (Chapagain et al., 2024; Lekshmi-Narayanan et al., 2024).

Self-Efficacy in Computer Programming

Self-efficacy, the belief that one can achieve positive results from one's actions (Bandura, 1977), is a good predictor of academic success in novice programming courses (Anastasiadou & Karakos, 2011; Bennedsen & Caspersen, 2019; Simon et al., 2019). Programming students' computer programming self-efficacy increases their persistence and effort and thus their academic achievement (Kanaparan et al., 2017; Tek et al., 2018).

Self-efficacy is not fixed; mastery experiences and regular feedback can increase it. Successive programming assignments can increase the self-efficacy of 1st-year students (Kinnunen & Simon, 2012). Women are more likely to experience this in traditional lecture courses than project-based learning courses, whereas men experience it in both (Jimenez et al., 2021). Research on the varying impact (by gender) of interventions to develop programming self-efficacy has been scant, but a previous study we conducted demonstrated that automated code-critiquing systems have greater impacts on the computer programming self-efficacy of women (Benjamin et al., 2025). Amato-Henderson and Sticklen (2025) found similar gendered impacts on academic motivation for students programming with the assistance of the WebTA autocritiquer (Ureel & Wallace, 2019).

Studies by Yilmaz and Karaoglan Yilmaz found that use of ChatGPT increased confidence (2023a), programming self-efficacy, and motivation (2023b) among Turkish undergraduate CS students learning object-oriented programming in Java, as measured by the Computer Programming Self-Efficacy Scale (CPSES; Ramalingam & Wiedenbeck, 1998) adapted into Turkish. But Johnson et al. (2024) found that American undergraduate students who used ChatGPT in an introductory agricultural systems technology course had less programming self-efficacy than students who programmed without GenAI help, as measured by an Arduino programming self-efficacy survey (Kittur, 2020). However, the scale they used was developed to measure programming self-efficacy of electrical and electronics engineering majors in the Indian context and may not be applicable to American CS students. A recent meta-analysis of experimental studies using ChatGPT in education concluded that its impact on students' self-efficacy was nonsignificant (Deng et al., 2025), but the authors pointed out the small number of studies on the topic and called for further research.

Our research team investigated the impact of AI-generated code explanations on 1st-year engineering and CS students' learning and programming self-efficacy at MTU. Because GPT provides the best line-by-line explanations, which are favored by novice coders (MacNeil et al., 2023), we leveraged the freely available Chat GPT-4.0 to assist students in making sense of code written in a language unknown to them. Specifically, we investigated two research questions (RQs):

RQ1: What is the impact of AI-generated code explanations on students' computer programming self-efficacy? Does this impact vary by discipline (CS vs. engineering) or gender identity?

RQ2: What are the impacts of this AI assistance on student learning and intent to use AI?

Method

Our research team employed a constructivist approach. Student explanations are an effective constructivist approach to learning to code (Chapagain et al., 2024). Novice programmers often need scaffolding around their first attempts at code explanation, which GenAI can support. This work includes a quantitative assessment of students' beliefs about their programming independence and persistence, as well as their ability to complete complex programming tasks, using Likert-scale questions from Ramalingam and Wiedenbeck's (1998) CPSES.

In a convergent mixed methods approach (Creswell & Plano Clark, 2017), we also employed open-ended questions to determine what students perceived they learned and to gather their reflections on their past use of GenAI and intentions for future use. As Reunanen and Nieminen noted, "a qualitative approach allows for a more in-depth exploration of the complexities and nuances of the impact of AI on programming education" (2024, p. 308). Indeed, Bond et al. (2024) called for more qualitative and mixed methods studies investigating the use of AI in higher education. The open-ended questions concerned (1) students' learning from the exercise, (2) previous AI use in coding, and (3) how they might leverage AI to learn and understand coding. This study was approved by MTU's Institutional Review Board administrator.

Participants

This study involved 297 1st-year engineering and CS students. At MTU, engineering students learn MATLAB programming in ENG1101 and expand their knowledge in ENG1102 in their 1st year. Those studying applied computing and electrical engineering learn the C language in CS1111. Software engineers, computer engineers, and other 1st-year CS majors are assessed on their programming skills and placed into CS1121, CS1122, or CS1131, where they learn to use Java. As Table 1 reflects, none of the participants had experience with FORTRAN.

Intervention Assignment and Assessment

Near the end of their introductory programming courses, students were asked to comment code in FORTRAN without the help of AI. They were then asked to have ChatGPT-4.0 provide code comments for them and to reflect on their learning, previous AI use, and thoughts on future use. Figure 1 shows the FORTRAN code and the six-step instructional prompt for this assignment provided to participants.

Table 1. Participant demographics by course number.

Variable	ENG1101	ENG1102	CS1111	CS1121	CS1131	Overall
Number of participants	147	22	72	29	27	297
Gender identity (percentage of respondents)						
Male	72.11	54.54	77.78	82.76	70.37	73.06
Female	23.13	40.91	18.05	6.90	11.11	20.54
Nonbinary/other/prefer not to say	4.76	4.55	4.17	10.34	18.52	6.40
Prior programming experience (percentage of respondents)						
Java	11.61	4.44	7.04	76.79	67.31	20.83
C	4.84	2.22	48.59	7.14	9.62	15.54
C++	2.90	8.89	11.97	0.00	0.00	4.96
Python	7.42	6.67	4.23	12.50	21.15	8.26
MATLAB	87.74	77.78	17.61	0.00	1.92	55.04
Scratch	1.29	0.00	2.82	1.79	7.69	2.15

Variable	ENG1101	ENG1102	CS1111	CS1121	CS1131	Overall
JavaScript	2.58	2.22	2.82	3.57	0.00	2.48
FORTRAN	0.00	0.00	0.00	0.00	0.00	0.00
Major (percentage of respondents)						
Computer-science-related major	2.05	0.00	6.94	65.52	74.07	15.88
Engineering-related major	91.78	86.36	84.72	17.24	18.52	75.67
Other	6.16	13.64	8.33	17.24	3.70	8.11
Undecided	0.00	0.00	0.00	0.00	3.70	0.34

Note. ENG = Engineering; CS = computer science.

```
program add_xy
    implicit none
    integer, parameter :: n = 100
    real :: x(n), y(n)
    real :: f(n, n)
    integer :: i, j
    character(len=20) :: filename
    write(*,*) 'Enter output file name:'
    read(*,*) filename

    ! Initialize x and y values
    do i = 1, n
        x(i) = i
        y(i) = i
    end do

    ! Calculate f(x, y) = x + y
    do i = 1, n
        do j = 1, n
            f(i, j) = x(i) + y(j)
        end do
    end do

    ! Open a file and write the array to it
    open(unit=10, file=filename, status='unknown')
    do i = 1, n
        do j = 1, n
            write(10, '(2I4, F10.2)') i, j, f(i, j)
        end do
    end do
    close(unit=10)

    write(*,*) 'File writing complete.'
end program add_xy
```

1. Please copy and paste the above code here. Add comments to this code and submit the commented code in the provided text box. Do not discuss this code with anyone, research it, or run it through AI. Just give us your best attempt at commenting. There is no right or wrong answer to this question. We are interested in studying the process by which students understand unfamiliar code. Please communicate your understanding.
2. Please explain in plain English what you think this code does. [text box provided]
3. Next, go to ChatGPT [embed hyperlink https://chat.openai.com/]and ask it "Can you please comment this FORTRAN code for me?" When it prompts you to, copy and paste the above code into the Chat GPT prompt. Read through the Chat GPT code comments but please don't go back and change your attempt at commenting the code above.
4. What did you learn from ChatGPT?
5. Have you ever used ChatGPT or other AI to help you understand or create code before this assignment? If so, how?
6. How might you use ChatGPT to help you learn and understand computer programming?

Figure 1. FORTRAN code and student instructions. AI = Artificial intelligence.

Students were asked to complete the survey items from the CPSES regarding their programming independence and persistence as well as their belief in their ability to complete complex programming tasks (Ramalingam & Wiedenbeck, 1998), both before and after the assignment. The CPSES uses a 7-point Likert scale (1 = *not confident at all*; 7 = *absolutely confident*) for responses to survey items. These CPSES items were administered:

Factor 1: [Programming] Independence and Persistence

1.1 Complete a programming project if I had a lot of time to complete the program.
1.2. Complete a programming project once someone else helped me get started.
1.3. Complete a programming project if I could call someone for help if I got stuck.
1.4. Complete a programming project if I had just the built-in help facility for assistance.
1.5. Complete a programming project if I had only the language reference manual for help.
1.6. Complete a programming project if someone showed me how to solve the problem.
1.7. Find ways of overcoming the problem if I got stuck at a point while working on a programming project.
1.8. Debug (correct all the errors) a long and complex program that I had written, and make it work.

Factor 2: Complex Programming Tasks

2.1. Understand the structure of programs.
2.2. Make use of a script that is already defined, given a clearly labeled declaration of the commands.
2.3. Identify the functions in the problem domain and declare, define, and use them.
2.4. Write a program that someone else could comprehend and add features to at a later date.
2.5. Make use of a pre-written function, given a clearly labeled declaration of the function.
2.6. Mentally trace through the execution of a long, complex, multi-file program given to me.
2.7. Rewrite lengthy confusing portions of code to be more readable and clear.

Data Analysis

The quantitative data were evaluated for change between the ordinal values of the pretest and posttest and descriptive statistics. We used sign tests for a measure of change with a p value of .05 as a threshold of significance and median levels and quartiles for descriptive statistics. Responses to the survey items on the CPSES were analyzed for the overall participant population, by field of study, as well as by gender. Sign tests were performed both across all participants in the study and among the different subgroups to see how reading code with AI affects self-efficacy.

Inductive coding was used to identify concepts within the reflective questions (Patton, 2015; Saldaña, 2016). Simultaneous coding was used where more than one code could be applied to the same passage of text simultaneously (Saldaña, 2016). Codes were then organized into larger themes for presentation (Patton, 2015).

Positionality

Positionality influences research across its lifecycle, making reflection on positionality essential to reducing bias in research (Secules et al., 2021). This team includes one undergraduate engineering student, two CS graduate students, two CS faculty, one faculty member jointly appointed in CS as well as psychology and human factors, and two engineering faculty. The team includes two women, five men, and one nonbinary person. Generation Z, Generation X, and the Silent Generation are represented. For some of our team members, digital technologies have been ubiquitous throughout their lives, while others have witnessed their gradual evolution and vividly remember life before the internet. Some remember slide rules and cut their teeth on mainframes before personal computers were ubiquitous. As a team, our members represent those willing to adopt and investigate the use of GenAI technologies within our teaching and apply a critical lens to its use in the future of education, grounding our pedagogy in evidence-based research.

Results

Results provided here include the quantitative analysis for the CPSES survey items, followed by the qualitative coding results for the open-ended survey questions.

CPSES Results

All the pretest and posttest factors exhibited a negative skew, speaking to a higher than average self-efficacy. The combined question score for Factor 1 for all subgroups showed significantly greater posttest values, indicating an increase in programming independence and persistence. For all subgroups, the combined questions for Factor 2 showed that posttest self-efficacy was significantly greater than pretest self-efficacy, indicating an increase in self-efficacy with complex programming tasks. Descriptive statistics for pretest and posttest values are presented in Table 2 for the Factor 1 response scores and in Table 3 for Factor 2 response scores, including mean scores for the first through third quartiles (Q1-3) and overall for each question.

The data were examined to determine any difference in results based on students' discipline of study (engineering majors, CS majors, and other majors). For Factor 1, engineering majors showed the exact same changes as the combined subgroup of all majors. CS majors exhibited greater change on Factor 1 than the combined subgroup. In addition, CS majors showed a significant increase in posttest scores for Question 1.5 on Factor 1 (complete a programming project with the reference manual for help). The other majors subgroup showed no evidence of significant change

across any of the questions or combined questions on Factor 1. For Factor 2, engineering majors differed from the combined subgroup, showing no evidence of change on Questions 2.1 (understanding problem structure) and 2.6 (mentally tracing complex programs increases in posttest scores. CS majors showed a significant increase in Question 2.1 of Factor 2 but no evidence of change in the combined questions.

The data were also investigated for any differences between male- and female-identified participants. Nonbinary, other, and prefer not to say participants did not show significant change for any question or combined question on both factors, but as the overall population of these respondents was low it is difficult to draw conclusions about this population. Female participants showed no evidence of change for the combined questions for Factor 1 but did show a significant increase in posttest scores for Questions 1.1 (complete a programming project given ample time), 1.5 (completing a program with help from the reference manual), 1.7 (overcoming programming problems), and 1.8 (debugging), and a significant decrease in posttest scores for Question 1.3 (completing programs with the ability to call someone for help). For male participants, Factor 1 showed a significant increase in posttest scores for the combined questions well as for Questions 1.1 and 1.7. Both Questions 1.3 and 1.6 (completing programs with someone showing them how) showed a significant decrease in posttest scores. These results are consistent with the combined subgroup analysis of all genders. Consistent with the combined subgroup, female participants showed a significant increase in posttest scores for the combined Factor 2 questions. This was not seen in male students. Within Factor 2, Questions 2.6 (tracing through complex programs) and 2.7 (rewriting confusing code) showed a significant increase in posttest scores for male participants, which is consistent with the combined subgroup.

Table 2. Factor 1: Programming independence and persistence.

Question	Pretest					Posttest					Significant change
	Q1	Q2	Q3	μ	*N*	Q1	Q2	Q3	μ	*N*	
1-8 Combined	5.25	5.87	6.25	5.75	293	5.25	5.87	6.38	5.75	281	.0074 ^
1	5	6	7	5.80	297	5	6	7	5.93	283	.000016 ^
2	6	6	7	6.14	296	6	6	7	6.05	283	–
3	6	7	7	6.35	296	6	6	7	6.21	283	.0069 *
4	5	6	7	5.70	297	5	6	6	5.55	283	–
5	4	5	6	4.95	295	4	5	6	5.01	282	–
6	6	7	7	6.31	297	6	6	7	6.11	283	.00014 *

Question	Pretest					Posttest					Significant change
	Q1	Q2	Q3	μ	*N*	Q1	Q2	Q3	μ	*N*	
7	5	6	6	5.49	297	5	6	7	5.73	282	.0000011 ^
8	5	5	6	5.23	297	5	6	6	5.37	283	.0073 ^

Note. Significant decrease (pre > post) denoted by *, significant increase (pre < post) denoted by ^, neither increase nor decrease denoted by –.

Table 3. Factor 2: Complex programming tasks.

Question	Pretest					Posttest					Significant change
	Q1	Q2	Q3	μ	*N*	Q1	Q2	Q3	μ	*N*	
1-7 Combined	4.57	5.28	6.00	5.25	296	4.71	5.57	6.00	5.57	283	.012 ^
1	5	5	6	5.37	297	5	6	6	5.44	283	.0076 ^
2	5	6	6	5.72	297	5	6	6	5.67	283	–
3	5	6	6	5.31	297	5	6	6	5.35	283	–
4	5	5	6	5.29	297	5	6	6	5.45	283	.028 ^
5	5	6	7	5.77	297	5	6	7	5.74	283	–
6	4	5	6	4.69	297	4	5	6	4.91	283	.016 ^
7	4	5	6	4.63	296	4	5	6	5.00	284	.0000011 ^

Note. Significant decrease (pre > post) denoted by *, significant increase (pre < post) denoted by ^, neither increase nor decrease denoted by –.

Coding Results From Open-Ended Question Responses

We begin with the results from the question "What did you learn?" Table 4 provides the percentage of responses for which each code was applied.

Table 4. Code results by course number for "What did you learn?"

Theme/Codes	% ENG1101 N = 147	% ENG1102 N = 22	%CS1111 N = 72	% CS1121 N = 29	% CS1131 N = 27	% Overall N = 297
Increased understanding	84.84	91.11	88.03	89.29	96.15	87.44
Check for confirmation	13.55	20.00	9.15	25.00	17.31	14.38
Check for misconceptions	13.23	4.44	7.04	16.07	13.46	11.40
Translatable experience	10.00	4.44	10.56	10.71	13.46	10.08
Did not increase understanding	7.74	4.44	5.63	5.36	1.92	6.28
No help	3.55	0.00	3.52	3.57	1.92	3.14
Omitted	2.58	2.22	1.41	1.79	0.00	1.98
Overwhelming	1.61	2.22	0.70	0.00	0.00	1.16
Code elements/commands						
Arrays	40.65	31.11	50.00	44.4	61.54	44.30
Output	27.74	13.33	28.17	26.79	42.31	27.93
Math	25.16	11.11	25.35	14.29	28.85	23.47
Files	21.29	13.33	20.42	12.50	40.38	21.32
Commenting	17.10	24.44	18.31	28.57	28.85	20.00
Variables	14.84	11.11	20.42	26.79	19.23	17.36
Loops	10.00	6.67	4.93	14.29	9.62	8.93
Commands	4.84	4.44	7.75	7.14	5.77	5.79

Note. ENG = Engineering; CS = computer science.

Overwhelmingly, participant responses (87.44%) showed that AI code explanations increased the students' self-perceived understanding of a language new to them. AI code comments simplified the code for them and helped them make sense of it. As one student wrote, "ChatGPT explained not just what the code was doing, but also why each step is important, which makes it easier to understand for someone unfamiliar with code." Participants indicated that the AI-provided comments confirmed something they suspected the code would do (14.38%), corrected a misconception they had (11.40%), or that it helped them see the transferability of programming experience between languages (10.08%).

The detailed responses about what novice programmers learned from using AI to explain code in an unknown language pertained to the ability to recognize and understand certain coding elements and commands, including arrays, output, math within a program, files, commenting, variables, loop structure, and other commands. Often, participants already understood this programming concept in another language and were highlighting new knowledge of its syntax in the new (to them) language. For example, one participant wrote, "I learned that the comment syntax for FORTRAN is ! comment and that arrays are defined using parentheses in FORTRAN."

Only 6.28% of the respondents indicated that AI did not increase their understanding of code in a new language. Their reasons were that they found the AI comments to be of no help at all, that AI omitted explanations of key syntax they did not understand, or that they found the AI-provided explanation overwhelming. However, these same respondents often added that explanations had increased their understanding, indicating that the AI was explaining part of the code well and follow-up questions could have clarified the remainder of the code. As one student wrote, the code did not "explain what the write() parameters mean or what the open() command does, which is what I don't really understand still. It did clear up what 'real' meant and it confirmed that x, y, and f are arrays." The students in this study were directed to provide a specific prompt and may not have felt they had the agency to ask these follow-up questions.

Table 5 provides a qualitative analysis from the results of the question, "Have you ever used ChatGPT or other AI to help you understand or create code before this assignment? If so, how?" A majority (57.85%) of respondents admitted to using AI, mostly for coding (56.53%). The percentage of respondents indicating they used AI for coding was greater in courses in which students possessed more coding experience. Students with transferable information-seeking experience may end up using AI to its fullest capabilities, as a student who sought help first in documentation centers and then turned to AI stated:

> I have used it for help with Arduino on MATLAB. This is mostly due to the fact that the documentation center for things like the LCD screen and rgb LED is not helpful at all. It helped me understand what function to call, what those functions do and even how to set up a circuit for that arduino component.

Table 5. Coding results by course number for "Have you ever used ChatGPT or other AI to help you understand or create code before this assignment? If so, how?"

Theme/Code	% ENG1101 N = 147	% ENG1102 N = 22	% CS1111 N = 72	% CS1121 N = 29	% CS1131 N = 27	% Overall N = 297
Yes	53.55	57.78	52.11	76.79	78.85	57.85
For coding	52.26	57.78	50.70	75.00	76.92	56.53
Not for coding	1.29	0.00	1.41	1.79	1.92	1.32
No	46.45	40.00	47.18	23.21	23.08	41.98
Reasons students used AI for coding						
Debugging	15.48	26.67	16.90	19.64	19.23	17.36
When confused	12.58	24.44	13.38	21.43	34.62	16.36
Code creation	10.00	15.56	11.27	12.50	23.08	12.07
Improving code	2.90	11.11	1.41	1.79	1.92	2.98
Why not?	8.06	2.22	1.41	0.00	7.69	5.29
Friends	1.29	0.00	0.70	0.00	1.92	0.99
Instructors	1.61	0.00	0.70	0.00	0.00	0.99
Doesn't help	3.23	2.22	0.00	0.00	1.92	1.98
Documentation	1.94	0.00	0.00	0.00	3.85	1.32

Note. AI = Artificial intelligence; ENG = engineering; CS = computer science.

Students indicated they leveraged AI to help them code in several ways. The biggest group, 17.36%, used AI to help them debug code. For both the engineering and CS course sequences,

responses indicating students were using AI correlated with students' level of experience programming. One student explained that they typically use AI for debugging:

> By asking ChatGPT to tell me how my code works. If it describes the code doing something that I don't want it to do, I will go to the piece of code and find out why and fix the problem. I like this way of using AI for coding because it doesn't just flat out give me the answer, it just points me in the direction of the problem so that I can come up with a solution on my own.

Students also reported using AI when confused about an assignment or code (16.36% overall) and in code creation (12.07%). Much as for debugging, courses where students had more programming experience also had more students reporting using AI for these purposes.

Table 6 shows the coding results for the question, "How might you use ChatGPT to help you learn and understand computer programming?" Two main themes emerged in these results: using AI as a pedagogical tool and using AI for help with coding tasks. Regarding using AI as a pedagogical tool, the largest group, 58.02% overall, said they might use AI to explain code to them, which is what the assignment in this study asked them to do. Many wrote that AI broke down explanations in a way that was easily digestible, especially for beginners.

This in turn, allowed for deeper understanding and comprehension:

> When not wanting a direct answer, I'd ask it to provide a step-by-step process on how to think/go about the problem without directly providing how to do those steps. I would use it to help define and break down computer programming terms, functions, etc. conceptually.

Students also (35.87% overall) indicated that they would use AI to develop their understanding. Although the two concepts are related, we coded using AI to explain code and developing understanding separately because the first pertains to the actions of AI, while the second is internal to the student. Among both engineering and CS courses, more students indicated they would use AI to develop their understanding in classes where more students had more programming experience than in novice programming classes. When leveraging AI as a pedagogical tool, 8.76% of respondents also indicated that they could ask it questions. ChatGPT responses are often organized through top-down design, beginning with a broad overview and then breaking explanations into component concepts, a feature that 6.78% of respondents mentioned. An additional 5.95% of respondents asserted that they found AI a useful tool for learning. Some students (3.80%) also valued ChatGPT for its natural language processing capabilities.

Table 6. Coding results by course number for the question, "How might you use ChatGPT to help you learn and understand computer programming?"

Theme/Code	%ENG1101 N = 147	%ENG1102 N = 22	%CS1111 N = 72	%CS1121 N = 29	%CS1131 N = 27	%Overall N = 297
AI as a pedagogical tool						
To explain code	55.81	62.22	58.45	66.07	57.69	58.02
To develop my understanding	34.19	35.56	36.62	37.50	42.31	35.87
Questions	9.68	11.11	6.34	8.93	7.69	8.76
Top-down design	7.10	6.67	4.23	7.14	11.54	6.78
Tool for learning	5.16	2.22	7.04	7.14	9.62	5.95

Theme/Code	%ENG1101 N = 147	%ENG1102 N = 22	%CS1111 N = 72	%CS1121 N = 29	%CS1131 N = 27	%Overall N = 297
Natural language	4.19	0.00	3.52	5.36	3.85	3.80
Coding tasks						
Debugging	23.23	17.78	22.54	14.29	26.92	22.15
Commands	15.16	15.56	12.68	12.50	7.69	13.72
Creating code	9.03	4.44	7.04	5.36	7.69	7.77
Commenting	8.39	0.00	8.45	5.36	5.77	7.27
Improving coding abilities	4.52	8.89	2.11	5.36	5.77	4.46
Improving code	4.19	2.22	2.82	3.57	3.85	3.64
Won't use it	3.23	4.44	7.04	5.36	3.85	4.46

Note. AI = Artificial intelligence; ENG = engineering; CS = computer science.

The second main theme that emerged in response to the question about how students might use ChatGPT to help them learn and understand programming was that it can assist students with coding tasks. Overall, the most common code in this theme was debugging (22.15% overall). When students had written code but could not figure out why it did not work, AI could assist them with debugging. Students' second most common response in this theme was that they would use AI to better understand specific commands within a programming language. A student described using AI to understand, explain, and solve their niche programming issue, which caused deeper understanding:

> Yes, when I get absolutely stuck and documentation or friends aren't working, I ask AI how to use a certain function or how to do something (usually in vague terms), then apply what I learn from the AI to my specific scenario. I also sometimes ask what an error I receive means.

While many faculty fear that students will use AI to cheat and be lazy (Denny et al., 2024; Petricini et al., 2024), only 7.27% of the overall respondents indicated they would use AI to create code for them. At the same time, only 4.46% of respondents indicated they would not use Chat GPT in the future. Clearly faculty can expect their students to be using AI.

Discussion of Results and Implications for Teaching

This work has shown that the use of Chat GPT-4.0 for code explanations can increase self-perceptions of programming independence and persistence as well as self-efficacy for complex programming tasks. This is consistent with other results showing that the use of GenAI can increase programming students' confidence (Yilmaz & Karaoglan Yilmaz, 2023a) and self-efficacy (Yilmaz & Karaoglan Yilmaz, 2023b). However, we found that only men showed results in programming independence and persistence, while only women showed results for completing complex programming tasks.

Programming Independence and Persistence

The results provide perspective on the effects of the use of AI for code explanation on students' help-seeking behaviors and programming self-efficacy. Although female students in the study displayed no significant change overall on Factor 1: Programming Independence and Persistence, they did show significant changes on Questions 1.1, 1.5, 1.7, and 1.8 and a significant decrease on Question 1.3. Prior work found code autocritiquing systems increased the self-efficacy of women regarding programming independence and persistence (Benjamin et al., 2025). Further research is needed to understand why GenAI use did not have these effects for women. When we examined the impact by discipline of study, both engineering and CS students exhibited an increase in self-efficacy with regard to programming independence and persistence.

One indication of greater self-perceived programming independence and persistence was that of less intent to rely on others for programming assistance, as both the quantitative and qualitative data evidence. Students' responses indicated a decrease in confidence for items related to completing programs if they could call someone for help (Question 1.3 of Factor 1 of the CPSES) or another person demonstrated how to solve a programming problem (Question 1.6). Question 1.3 displayed significant decreases for both women and men while only men displayed a significant decrease for Question 1.6. However, relying less on other people may not benefit students in the long run, given other people's significant role as a facilitating condition in technology adoption, including the adoption of programming languages (Jarvie-Eggart, et al., 2021, 2024). Students' information-seeking behavior about programming may be shifting away from humans and toward AI. Within the student responses to the question about their past use of ChatGPT for programming, only 1.98% of respondents indicated that they would prefer to use other people over ChatGPT. Xue et al. (2024) found that students allowed to use ChatGPT-3.5 as a resource for an introductory programming class used traditional educational resources less. Instructors leveraging AI for learning may want to ameliorate the potential loss of social connection among their students by also promoting the formation of study groups and the use of learning centers.

The current study's results indicate a shift in students' intentions away from using AI to write code for them and toward using it for explanation and thus learning. While 12.07% had previously used AI for code creation, only 7.77% wanted to use it for this purpose in the future. Erez et al. (2025) showed that proper guided use in class shifted novice programming students' views about using GenAI toward a positive view of it as a learning tool. Instructors should introduce AI as a pedagogical tool, guiding students in their use of AI, modeling for their students how to use AI to develop as programmers. Students should be taught to view GenAI feedback with a critical eye.

Both the qualitative and quantitative data provide evidence of an increase in debugging self-efficacy through the use of AI. Question 1.8 of Factor 1 of the CPSES also showed a statistically significant increase in students' self-efficacy beliefs around debugging for the whole participant population and for female students, but not male students. Debugging was also the most often mentioned purpose for which students have used AI for coding in the past (17.46% of all respondents) and intend to use it for coding in the future (22.15%). However, the use of AI for debugging is controversial. Research shows it benefits students who are able to identify and ignore incorrect code suggestions but increases metacognitive difficulties for students who are already struggling (Prather et al., 2024). Truly novice programmers will have difficulty identifying AI suggestions to disregard. Effectively guiding students through the appropriate use of AI for debugging while minimizing metacognitive difficulties will require further research, including analysis of the impact of debugging assistance on female students' programming confidence.

Complex Programming Tasks

Factor 2 of the CPSES, Complex Programming Tasks, exhibited a significant increase in self-efficacy among the overall populations studied. Our work corroborates the work of Reunanen and Nieminen (2024), who found that GenAI increased 1st-year programming students' ability to handle more complex programming tasks than students in an introductory class otherwise could. When we examined the impacts of GenAI use by gender, women showed improvement in their confidence to successfully complete complex programming tasks, with improvement on Questions 2.1, 2.4, 2.6, and 2.7, but men did not. Although men did not show an increase in the combined Factor 2 questions, they did display an increase in Questions 2.6 and 2.7. When examining the data by area of study, we found that engineering students exhibited an increase in confidence in complex programming tasks while CS majors displayed no change. Thus, GenAI use may offer particular benefits when teaching programming to non-CS majors, who may lack programming self-efficacy as it is not their primary major. In fact, a study found that 1st-year engineering students experienced frustration more than any other emotion when learning programming (Atiq & Loui, 2022).

The responses to Factor 2 show that students displayed more confidence in their abilities to understand program structures, trace through programs, write programs, and improve programs. Question 2.1 of Factor 2 pertained to understanding the structure of programs, which showed a significant increase for both women specifically and the students overall. However, Question 2.6 aligned with Question 2.1, asking about students' confidence in their ability to mentally trace through complex multifile code. Both men and women showed an increase in self-efficacy for this question. Question 2.7 increased significantly overall and for both women and men. Both of these questions address whether students understand what code does. Increases in responses to Questions 2.1 and 2.2 of Factor 2 align with the qualitative results in which students reported increased understanding of programming from using Chat GPT and intended to use it in the future to further develop their understanding.

After breaking the results down by gender, we found that these impacts on Factor 2 were significant for women, but not men. Other work has shown that digital resources (such as autocritiquers) can be useful to increase women's programming self-efficacy with regard to complex programming tasks (Benjamin et al., 2025). Another study of Malaysian undergraduate students found low self-efficacy among students when solving complex programming tasks on their own, but increased self confidence in tackling these problems with guidance from another person or digital resources (Thinakaran et al., 2024). These results, alongside our own, indicate that technology-assisted learning may ameliorate gender differences in programming self-efficacy with regard to complex programming tasks. This is important because women typically have lower levels of programming self-efficacy (Beyer, 2014; Cohoon, 2006), despite similar programming abilities (Galpin et al., 2003).

Limitations

This work relied on self-reported use of GenAI and reports of intentions to use AI, with the attending limitation as to accuracy. Future work could monitor students' actual use rather than rely on self reports. Additionally, the study was run at a single, predominantly White institution; therefore the results may not generalize to other student populations.

Conclusion

We considered whether ChatGPT-4.0 is a suitable tool to generate explanations students can use to further their understanding of code. Our results showed significant improvements in self-efficacy for the participants with regards to programming independence and persistence as well as complex programming tasks (Factors 1 and 2 of the CPSES). Both engineering and CS majors showed improvements on Factor 1, while only engineers exhibited significant improvement in their confidence related to Factor 2. Future work should explore the promise of GenAI-assisted learning in increasing the abilities of non-CS students to program independently. Only male students displayed significant increases on Factor 1 while only female students showed significant increases in confidence regarding Factor 2. Although AI code explanations may increase academic success, further research is needed to examine how and why the impacts of these explanations vary by gender.

The qualitative results show promising trends, suggesting instructors should introduce GenAI as a pedagogical tool. Students may be more likely to use GenAI as they gain programming experience. The exercise used in this study, guiding students through the use of GenAI to explain code, shifted students' views to its use as a learning assistant, rather than a tool that writes programs for them. Thus, we recommend that faculty be trained in how to guide students through the appropriate use of GenAI.

Acknowledgments

The authors thank Mr. Ken Thiemann for implementing a pilot version of this study in his summer course, as well as Laura Albrant and Jyoti Suhag for contributing to qualitative data analysis strategy.

References

Amato-Henderson, S. L., & Sticklen, J. (2025). *WIP: Computation and student engagement in first-year engineering* [Paper presentation]. 2025 ASEE Annual Conference & Exposition, Montreal, Quebec, Canada. https://doi.org/10.18260/1-2–57387

Anastasiadou, S. D., & Karakos, A. S. (2011). The beliefs of electrical and computer engineering students' regarding computer programming. *The International Journal of Technology, Knowledge, and Society*, *7*(1), 37–51. https://doi.org/10.18848/1832-3669/CGP/v07i01/56170

Atiq, Z., & Loui, M. C. (2022). A qualitative study of emotions experienced by first-year engineering students during programming tasks. *ACM Transactions on Computing Education*, *22*(3), Article 32. https://doi.org/10.1145/3507696

Bandura, A. (1977). Self-efficacy: Toward a unifying theory of behavioral change. *Psychological Review*, *84*(2), 191–215. https://doi.org/10.1037/0033-295X.84.2.191

Banjade, R., Oli, P., Sajib, M. I., & Rus, V. (2024). Identifying gaps in students' explanations of code using LLMs. In A. M. Olney, I.-A. Chounta, Z. Liu, O. C. Santos, & I. I. Bittencourt (Eds.), *Artificial intelligence in education* (pp. 268–275). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-64299-9_21

Benjamin, M., Albrant, L., Jarvie-Eggart, M. E., Ureel, L. C., Brown, L. E., Sticklen, J., & Hamlin, A. J. (2025, February 9). *Gendered impacts of code critiquers on self-efficacy in first-year engineering students* [Paper presentation]. 2025 Collaborative Network for Engineering & Computing Diversity, San Antonio, TX. https://peer.asee.org/54094

Bennedsen, J., & Caspersen, M. E. (2019). Failure rates in introductory programming—12 years later. *ACM Inroads*, *10*(2), 30–35. https://doi.org/10.1145/3324888

Beyer, S. (2014). Why are women underrepresented in computer science? Gender differences in stereotypes, self-efficacy, values, and interests and predictors of future CS course-taking and grades. *Computer Science Education*, *24*(2), 153–192. https://doi.org/10.1080/08993408.2014.963363

Bond, M., Khosravi, H., De Laat, M., Bergdahl, N., Negrea, V., Oxley, E., Pham, P., Chong, S. W., & Siemens, G. (2024). A meta systematic review of artificial intelligence in higher education: A call for increased ethics, collaboration, and rigour. *International Journal of Educational Technology in Higher Education*, *21*(1), Article 4. https://doi.org/10.1186/s41239-023-00436-z

Chapagain, J., Sajib, M. I., Prodan, R., & Rus, V. (2024). A study of LLM generated line-by-line explanations in the context of conversational program comprehension tutoring systems. In R. Ferreira Mello, N. Rummel, I. Jivet, G. Pishtari, & J. A. Ruipérez Valiente (Eds.), *Technology enhanced learning for inclusive and equitable quality education* (pp. 64–74). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-72315-5_5

Cohoon, J. M. (2006). Just get over it or just get on with it: Retaining women in undergraduate computing. In J. M. Cohoon & W. Asprey (Eds.), *Women and information technology: Research on underrepresentation*. MIT Press. https://doi.org/10.7551/mitpress/7272.003.0009

Creswell, J. W., & Plano Clark, V. L. (2017). *Designing and conducting mixed methods research* (3rd ed.). SAGE Publications.

Cunningham, K., Qiao, Y., Feng, A., & O'Rourke, E. (2022). Bringing "high-level" down to earth: Gaining clarity in conversational programmer learning goals. In *Proceedings of the 53rd ACM Technical Symposium on Computer Science Education* (Vol. 1, pp. 551–557). Association for Computing Machinery. https://doi.org/10.1145/3478431.3499370

Deng, R., Jiang, M., Yu, X., Lu, Y., & Liu, S. (2025). Does ChatGPT enhance student learning? A systematic review and meta-analysis of experimental studies. *Computers & Education*, *227*, Article 105224. https://doi.org/10.1016/j.compedu.2024.105224

Denny, P., Prather, J., Becker, B. A., Finnie-Ansley, J., Hellas, A., Leinonen, J., Luxton-Reilly, A., Reeves, B. N., Santos, E. A., & Sarsa, S. (2024). Computing education in the era of generative AI. *Communications of the ACM*, *67*(2), 56–67. https://doi.org/10.1145/3624720

Deriba, F. G., Sanusi, I. T., & Sunday, A. O. (2024). Enhancing computer programming education using ChatGPT—Mini review. In A. Müling & I. Jormanainen (Eds.), *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research* (Article 45, pp. 1–2). Association for Computer Machinery. https://doi.org/10.1145/3631802.3631848

Diaz, N., Roy, S., & Beltran, J. (2025). Exploring undergraduate AI perceptions: Knowledge, enthusiasm, and concerns. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education* (Vol. 2, p. 1753). Association for Computing Machinery. https://doi.org/10.1145/3641555.3705031

Edwards, J. M., Fulton, E. K., Holmes, J. D., Valentin, J. L., Beard, D. V., & Parker, K. R. (2018). *Separation of syntax and problem solving in Introductory Computer Programming* [Paper presentation]. 2018 IEEE Frontiers in Education Conference, San Jose, CA. https://doi.org/10.1109/FIE.2018.8658852

Erez, Y., Ayali, L., & Hazzan, O. (2025). Evolution of students' attitudes towards the use of generative AI tools in a CS1 course: Implications for instructors. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education* (Vol. 2, p. 1740). Association for Computing Machinery. https://doi.org/10.1145/3641555.3705064

Fan, Y., Tang, L., Le, H., Shen, K., Tan, S., Zhao, Y., Shen, Y., Li, X., & Gašević, D. (2025). Beware of metacognitive laziness: Effects of generative artificial intelligence on learning motivation, processes, and performance. *British Journal of Educational Technology*, *56*, 489–530. https://doi.org/10.1111/bjet.13544

Finnie-Ansley, J., Denny, P., Becker, B. A., Luxton-Reilly, A., & Prather, J. (2022). The robots are coming: Exploring the implications of OpenAI Codex on introductory programming. In J. Sheard & P. Denny (Eds.), *Proceedings of the 24th Australasian Computing Education Conference* (pp. 10–19). Association for Computing Machinery. https://doi.org/10.1145/3511861.3511863

Fowler, M., Hallstrom, J., Hollingsworth, J., Kraemer, E., Sitaraman, M., Sun, Y.-S., Wang, J., & Washington, G. (2022). Tool-aided learning of code reasoning with abstraction in the CS curriculum. *Informatics in Education*, *20*(4), 533–566. https://doi.org/10.15388/infedu.2021.24

Galpin, V. C., Sanders, I., Turner, H., & Venter, B. (2003). *Gender and educational background and their effect on computer self-efficacy and perceptions* (Technical Report TR-Wits-CS-2003-0). School of Computer Science, University of the Witwatersrand. Accessed May 30, 2025, at https://homepages.inf.ed.ac.uk/vgalpin1/pubs/GSTV03a.html

Geng, C., Zhang, Y., Pientka, B., & Si, X. (2023). *Can ChatGPT pass an introductory level functional language programming course?* arXiv. https://doi.org/10.48550/arXiv.2305.02230

Griffin, J. M. (2016). Learning by taking apart: Deconstructing code by reading, tracing, and debugging. In *Proceedings of the 17th Annual Conference on Information Technology Education* (pp. 148–153). Association for Computing Machinery. https://doi.org/10.1145/2978192.2978219

Handa, K., Bent, D., Tamkin, A., McCain, M., Durmus, E., Stern, M., Schiraldi, M., Huang, S., Ritchie, S., Syverud, S., Jagadish, K., Vo, M., Bell, M., & Ganguli, D. (2025, April 8). *Anthropic education report: How university students use Claude.* Anthropic. https://www.anthropic.com/news/anthropic-education-report-how-university-students-use-claude

Herden, O. (2024). Integration of chatbots for generating code into introductory programming courses. In *The Future of Education International Conference*, 14th ed., Florence, Italy. https://conference.pixel-online.net/library_scheda.php?id_abs=6673

Holstein, K., Wortman Vaughan, J., Daumé, H., Dudik, M., & Wallach, H. (2019). Improving fairness in machine learning systems: What do industry practitioners need? In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Article 600, pp. 1–16). Association for Computing Machinery. https://doi.org/10.1145/3290605.3300830

Hunt, A., & Thomas, D. (1999). *The pragmatic programmer: From journeyman to master.* Addison-Wesley.

Jacques, L. (2023). Teaching CS-101 at the dawn of ChatGPT. *ACM Inroads*, *14*(2), 40–46. https://doi.org/10.1145/3595634

Jarvie-Eggart, M., Owusu-Ansah, A., & Stockero, S. L. (2021, October 13–16). *Factors motivating engineering faculty to adopt and teach new engineering technologies.* 2021 IEEE Frontiers in Education Conference, Lincoln, NE. https://doi.org/10.1109/FIE49875.2021.9637235

Jarvie-Eggart, M., Stockero, S. L., & Owusu-Ansah, A. (2024). Factors influencing faculty's adoption of engineering technology: A qualitative study. *Computers and Education Open*, *7*, Article 100221. https://doi.org/10.1016/j.caeo.2024.100221

Jimenez, P. P., Pascual, J., Espinoza, J., Martin, S. S., & Guidi, F. (2021). Pedagogical innovations with a gender approach to increase computer programming self-efficacy in engineering students. In *2021 IEEE Global Engineering Education Conference* (pp. 322–328). IEEE. https://doi.org/10.1109/EDUCON46332.2021.9453933

Johnson, D. M., Doss, W., & Estepp, C. M. (2024). Using ChatGPT with novice Arduino programmers: Effects on performance, interest, self-efficacy, and programming ability. *Journal of Research in Technical Careers*, *8*(1). https://doi.org/10.9741/2578-2118.1152

Kanaparan, G., Cullen, R., & Mason, D. (2017). Self-efficacy and behavioural engagement in introductory programming courses. In *PACIS 2017 Proceedings* (Article 209). Association for Information Systems. https://aisel.aisnet.org/pacis2017/209

Kasneci, E., Sessler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gasser, U., Groh, G., Günnemann, S., Hüllermeier, E., Krusche, S., Kutyniok, G., Michaeli, T., Nerdel, C., Pfeffer, J., Poquet, O., Sailer, M., Schmidt, A., Seidel, T., … Kasneci, G. (2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, *103*, Article 102274. https://doi.org/10.1016/j.lindif.2023.102274

Kazemitabaar, M., Chow, J., Ma, C. K. T., Ericson, B. J., Weintrop, D., & Grossman, T. (2023). Studying the effect of AI code generators on supporting novice learners in introductory programming. In A. Schmidt, K. Väänänen, T. Goyal, P. O. Kristensson, A. Peters, S. Mueller, J. R. Williamson, & M. L. Wilson (Eds.), *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Article 45). Association for Computing Machinery. https://doi.org/10.1145/3544548.3580919

Kiesler, N., & Schiffner, D. (2023). *Large language models in introductory programming education: ChatGPT's performance and implications for assessments*. arXiv. https://doi.org/10.48550/arXiv.2308.08572

Kinnunen, P., & Simon, B. (2012). My program is OK—Am I? Computing freshmen's experiences of doing programming assignments. *Computer Science Education*, *22*(1), 1–28. https://doi.org/10.1080/08993408.2012.655091

Kittur, J. (2020). Measuring the programming self-efficacy of electrical and electronics engineering students. *IEEE Transactions in Engineering, 63*(3), 216–223. https://doi.org/10.1109/TE.2020.2975342

Koedinger, K. R., Booth, J. L., & Klahr, D. (2013). Instructional complexity and the science to constrain it. *Science*, *342*(6161), 935–937. https://doi.org/10.1126/science.1238056

Kuramitsu, K., Obara, Y., Sato, M., & Obara, M. (2023). KOGI: A seamless integration of ChatGPT into Jupyter environments for programming education. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on SPLASH-E* (pp. 50–59). Association for Computing Machinery. https://doi.org/10.1145/3622780.3623648

Leinonen, J., Denny, P., MacNeil, S., Sarsa, S., Bernstein, S., Kim, J., Tran, A., & Hellas, A. (2023). Comparing code explanations created by students and large language models. In *Proceedings of the 2023 Conference on Innovation and Technology in Computer Science Education* (Vol. 1, pp. 124–130). Association for Computing Machinery. https://doi.org/10.1145/3587102.3588785

Lekshmi-Narayanan, A. B., Oli, P., Chapagain, J., Hassany, M., Banjade, R., Brusilovsky, P., & Rus, V. (2024). Explaining code examples in introductory programming courses: LLM vs humans. *Proceedings of Machine Learning Research*, *257*, 107–117.

Lister, R., Fidge, C., & Teague, D. (2009). Further evidence of a relationship between explaining, tracing and writing skills in introductory programming. In *Proceedings of the 14th Annual ACM SIGCSE Conference on Innovation and Technology in Computer Science Education* (pp. 161–165). Association for Computing Machinery. https://doi.org/10.1145/1562877.1562930

Lister, R., Simon, B., Thompson, E., Whalley, J. L., & Prasad, C. (2006). Not seeing the forest for the trees: Novice programmers and the SOLO taxonomy. *SIGCSE Bulletin*, *38*(3), 118–122. https://doi.org/10.1145/1140123.1140157

Lo, C. K. (2023). What is the impact of ChatGPT on education? A rapid review of the literature. *Education Sciences*, *13*(4), Article 4. https://doi.org/10.3390/educsci13040410

Luckin, R., Holmes, W., Griffiths, M., & Forcier, L. B. (2016). *Intelligence unleashed: An argument for AI in education*. Pearson. https://www.pearson.com/content/dam/corporate/global/pearson-dot-com/files/innovation/Intelligence-Unleashed-Publication.pdf

MacNeil, S., Tran, A., Hellas, A., Kim, J., Sarsa, S., Denny, P., Bernstein, S., & Leinonen, J. (2023). Experiences from using code explanations generated by large language models in a web software development e-book. In *Proceedings of the 54th ACM Technical Symposium on Computer*

Science Education (Vol. 1, pp. 931–937). Association for Computing Machinery. https://doi.org/10.1145/3545945.3569785

Mah, D.-K., & Groß, N. (2024). Artificial intelligence in higher education: Exploring faculty use, self-efficacy, distinct profiles, and professional development needs. *International Journal of Educational Technology in Higher Education*, *21*(1), Article 58. https://doi.org/10.1186/s41239-024-00490-1

Margulieux, L.E., Morrison, B.B., & Decker, A. (2020). Reducing withdrawal and failure rates in introductory programming with subgoal labeled worked examples. *International Journal of STEM Education*, *7*, Article 19. https://doi.org/10.1186/s40594-020-00222-7

McCauley, R., Fitzgerald, S., Lewandowski, G., Murphy, L., Simon, B., Thomas, L., & Zander, C. (2008). Debugging: A review of the literature from an educational perspective. *Computer Science Education*, *18*(2), 67–92. https://doi.org/10.1080/08993400802114581

Murphy, L., Fitzgerald, S., Lister, R., & McCauley, R. (2012). Ability to "explain in plain English" linked to proficiency in computer-based programming. In *Proceedings of the Ninth Annual International Conference on International Computing Education Research* (pp. 111–118). Association for Computing Machinery. https://doi.org/10.1145/2361276.2361299

OpenAI. (2025). *ChatGPT* (May 2025 version) [Large language model]. https://chat.openai.com/chat

Patton, M. Q. (2015). *Qualitative research & evaluation methods: Integrating theory and practice* (4th ed.). SAGE Publications.

Petricini, T., Zipf, S., & Wu, C. (2024). Perceptions about generative AI and ChatGPT use by faculty and students. *Transformative Dialogues: Teaching and Learning Journal*, *17*(2), Article 2. https://doi.org/10.26209/td2024vol17iss21825

Prather, J., Reeves, B. N., Leinonen, J., MacNeil, S., Randrianasolo, A. S., Becker, B. A., Kimmel, B., Wright, J., & Briggs, B. (2024). The widening gap: The benefits and harms of generative AI for novice programmers. In *Proceedings of the 2024 ACM Conference on International Computing Education Research* Vol. 1, (pp. 469–486). Association for Computing Machinery. https://doi.org/10.1145/3632620.3671116

Ramalingam, V., & Wiedenbeck, S. (1998). Development and validation of scores on a computer programming self-efficacy scale and group analyses of novice programmer self-efficacy. *Journal of Educational Computing Research*, *19*(4), 367–381.

Reunanen, T, & Nieminen, N.. (2024). Artificial intelligence as a catalyst: A case study on adaptive learning in programming education. *In: Vesa Salminen (eds) Human Factors, Business Management and Society*. AHFE (2024) International Conference. AHFE Open Access, vol 135, (pp. 305-312). Applied Human Factors and Ergonomics International. http://doi.org/10.54941/ahfe1004957.

Roll, I., & Wylie, R. (2016). Evolution and revolution in artificial intelligence in education. *International Journal of Artificial Intelligence in Education*, *26*(2), 582–599. https://doi.org/10.1007/s40593-016-0110-3

Saldaña, J. (2016). *The coding manual for qualitative researchers* (3rd ed.). SAGE.

Sarsa, S., Denny, P., Hellas, A., & Leinonen, J. (2022). Automatic generation of programming exercises and code explanations using large language models. In *Proceedings of the 2022 ACM Conference on International Computing Education Research* (Vol. 1, pp. 27–43). Association for Computing Machinery. https://doi.org/10.1145/3501385.3543957

Scholl, A., Schiffner, D., & Kiesler, N. (2024). *Analyzing chat protocols of novice programmers solving introductory programming tasks with ChatGPT*. arXiv. https://doi.org/10.48550/arXiv.2405.19132

Secules, S., McCall, C., Mejia, J. A., Beebe, C., Masters, A. S., L. Sánchez-Peña, M., & Svyantek, M. (2021). Positionality practices and dimensions of impact on equity research: A collaborative inquiry and call to the community. *Journal of Engineering Education*, *110*(1), 19–43. https://doi.org/10.1002/jee.20377

Shawar, B. A., & Atwell, E. (2007). Chatbots: Are they really useful? *Journal for Language Technology and Computational Linguistics*, *22*(1), Article 1. https://doi.org/10.21248/jlcl.22.2007.88

Sheard, J., Carbone, A., Lister, R., Simon, B., Thompson, E., & Whalley, J. L. (2008). Going SOLO to assess novice programmers. In *Proceedings of the 13th Annual Conference on Innovation and Technology in Computer Science Education* (pp. 209–213). Association for Computing Machinery. https://doi.org/10.1145/1384271.1384328

Simon, Luxton-Reilly, A., Ajanovski, V. V., Fouh, E., Gonsalvez, C., Leinonen, J., Parkinson, J., Poole, M., & Thota, N. (2019). Pass rates in introductory programming and in other STEM disciplines. In *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education* (pp. 53–71). Association for Computing Machinery. https://doi.org/10.1145/3344429.3372502

Styve, A., Virkki, O. T., & Naeem, U. (2024). Developing critical thinking practices interwoven with generative AI usage in an introductory programming course. In *2024 IEEE Global Engineering Education Conference Proceedings*. IEEE. https://doi.org/10.1109/EDUCON60312.2024.10578746

Tek, F. B., Benli, K. S., & Deveci, E. (2018). Implicit theories and self-efficacy in an introductory programming course. *IEEE Transactions on Education*, *61*(3), 218–225. https://doi.org/10.1109/TE.2017.2789183

Thinakaran, R., Chuprat, S., Varadarajan, V., Yussubaliyeva, M., & Maqsood, M. (2024). Self-efficacy as students' motivation factor in learning programming. *Journal of Infrastructure, Policy and Development*, *8*(16), Article 16. https://doi.org/10.24294/jipd9252

Ullah, Z., Lajis, A., Jamjoom, M., Altalhi, A., & Saleem, F. (2020). Bloom's taxonomy: A beneficial tool for learning and assessing students' competency levels in computer programming using empirical analysis. *Computer Applications in Engineering Education*, *28*(6), 1628–1640. https://doi.org/10.1002/cae.22339

Ureel, L. C., II, & Wallace, C. (2019). Automated critique of early programming antipatterns. In *Proceedings of the 50th ACM Technical Symposium on Computer Science Education* (pp. 738–744). Association for Computing Machinery. https://doi.org/10.1145/3287324.3287463

Venables, A., Tan, G., & Lister, R. (2009). A closer look at tracing, explaining and code writing skills in the novice programmer. In *Proceedings of the Fifth International Workshop on Computing Education Research Workshop* (pp. 117–128). Association for Computing Machinery. https://doi.org/10.1145/1584322.1584336

Wermelinger, M. (2023). Using GitHub Copilot to solve simple programming problems. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education* (Vol. 1, pp. 172–178). Association for Computing Machinery. https://doi.org/10.1145/3545945.3569830

Winnie, D. (2021). *Essential Java for AP CompSci*. Apress. https://doi.org/10.1007/978-1-4842-6183-5_1

Xie, B., Loksa, D., Nelson, G. L., Davidson, M. J., Dong, D., Kwik, H., Tan, A. H., Hwa, L., Li, M., & Ko, A. J. (2019). A theory of instruction for introductory programming skills. *Computer Science Education*, *29*(2–3), 205–253. https://doi.org/10.1080/08993408.2019.1565235

Xue, Y., Chen, H., Bai, G. R., Tairas, R., & Huang, Y. (2024). Does ChatGPT help with introductory programming? An experiment of students using ChatGPT in CS1. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training* (pp.

331–341). Association for Computing Machinery. https://doi.org/10.1145/3639474.3640076

Yilmaz, R., & Karaoglan Yilmaz, F. G. (2023a). Augmented intelligence in programming learning: Examining student views on the use of ChatGPT for programming learning. *Computers in Human Behavior: Artificial Humans*, *1*(2), Article 100005. https://doi.org/10.1016/j.chbah.2023.100005

Yilmaz, R., & Karaoglan Yilmaz, F. G. (2023b). The effect of generative artificial intelligence (AI)-based tool use on students' computational thinking skills, programming self-efficacy and motivation. *Computers and Education: Artificial Intelligence*, *4*, Article 100147. https://doi.org/10.1016/j.caeai.2023.100147

Zi, Y., Li, L., Guha, A., Anderson, C. J., & Feldman, M. Q. (2025). "I would have written my code differently": Beginners struggle to understand LLM-generated code. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering* (pp. 1479–1488. Association for Computing Machinery. https://doi.org/10.1145/3696630.3731663